

Đặt học sâu vào đâu trong một hệ SLAM (Where Learning Fits in a SLAM System)

PHỤ THUỘC B/03-giai-phau-he-thong (đọc một hệ thống bằng luồng dữ liệu và ranh giới); E/24-thi-nghiem-va-ablation (sản nhiễu và oracle experiment — chương này dùng lại cả hai); B/11-gioi-han-tinh-toan cùng D/21-inference-va-trien-khai cho công ngân sách; B/08-dich-chuyen-phan-phoi cho công thứ tư.

TÓM TẮT

Chương này không dạy một phương pháp nào. Nó dựng bộ lọc để bạn quyết định phương pháp nào đáng đọc tiếp. Bạn sẽ có bốn công cụ: năm chỗ có thể cắm mạng vào một hệ SLAM cổ điển, một khung bốn cổng để loại sớm, một bản đồ từ bốn nỗi đau sang nhóm phương pháp, và một quy trình đọc paper trong ba mươi phút. Đọc xong bạn phát biểu được câu “chưa đáng” kèm con số thay vì kèm cảm giác, và biết đầu tư ba tháng tới vào đúng một chỗ. Nếu chỉ nhớ một điều: đo trần đóng góp của một khối trước, rồi mới đi tìm mạng để thay khối đó — và ưu tiên những chỗ mà khi hỏng thì hệ thống hỏng *ồn ào*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình học sâu** — một hàm có rất nhiều tham số, được đi tìm bằng cách cho xem nhiều ví dụ có đáp án. Trong chương này nó luôn xuất hiện như một *khối* nhận ảnh và trả về một đại lượng: điểm đặc trưng, độ sâu, mặt nạ, hoặc một pose.
- **suy luận (inference)** — một lần chạy mô hình đã huấn luyện trên một đầu vào mới. SLAM chạy suy luận trên từng khung hình, mỗi lần đúng một ảnh, nên nó bị chặn bởi *độ trễ* chứ không phải bởi *thông lượng*.
- **batch** — số mẫu được đưa qua mạng cùng lúc. GPU đạt hiệu suất cao khi batch lớn; SLAM thời gian thực buộc batch bằng 1, nên phần lớn con số FPS trong paper không áp dụng được cho bạn.
- **phân phối huấn luyện** — tập các tình huống mà mô hình đã được xem trong lúc huấn luyện. “Ra ngoài phân phối” nghĩa là gặp tình huống khác hẳn — đêm tối, mưa, hành lang không vãn, ảnh mờ — và đó là chỗ mọi bảo đảm về độ chính xác biến mất.
- **ma trận thông tin** — nghịch đảo của ma trận hiệp phương sai; trong một đồ thị nhân tử nó chính là *trọng số* của một phép đo. Đặt nó quá lớn nghĩa là bảo bộ tối ưu “hãy tin phép đo này”, và đó là cách hỏng âm thầm nhất của một hệ visual-inertial.
- **APE và RPE** — APE là sai số tuyệt đối của quỹ đạo sau khi căn chỉnh với chân trị; RPE là sai số tương đối trên các đoạn ngắn. APE nhạy với trôi tích lũy và với vòng đóng; RPE nhạy với chất lượng odometry cục bộ.
- **tỉ lệ hoàn thành chuỗi** — phần trăm khung hình mà hệ thống còn bám được. Nó phải luôn được báo cùng APE, vì một hệ mất bám giữa chừng chỉ còn được chấm trên đoạn dễ và vì thế có APE rất đẹp.
- **trần đóng góp (oracle ceiling)** — mức cải thiện tối đa mà một khối có thể mang lại, đo bằng cách thay khối đó bằng chân trị rồi chạy cả hệ. Nếu một bộ ghép cặp hoàn hảo chỉ giảm APE được một lượng c , thì mọi bộ ghép cặp học được cũng bị chặn trên bởi đúng c . Giá trị của c là thứ bạn phải tự đo trên hệ của mình; không mượn được của ai và không đọc được từ bảng của ai.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn thay DBoW bằng một mô hình nhận dạng địa điểm hiện đại. Recall@1 tăng vọt, nhưng số vòng đồng thành công không đổi một cái nào. Chuyện gì đã xảy ra, và lẽ ra bạn phải đo gì trước?
- Q2. Một paper báo APE thấp hơn ORB-SLAM3 nhiều lần. Ba câu hỏi nào phải hỏi trước khi tin, và câu nào loại được nhiều paper nhất chỉ trong ba phút?
- Q3. Vì sao một phép đo sai mà “tự tin” lại nguy hiểm hơn hẳn việc không có phép đo nào — trong khi với một bộ phân lớp ảnh thì hai chuyện đó gần như tương đương?
- Q4. Một paper báo mạng chạy rất nhanh trên một GPU để bàn. Vì sao con số đó gần như không nói gì về việc nó có chạy nổi 20 Hz trên Jetson Orin hay không?
- Q5. Bạn có ba tháng và bốn nỗi đau. Phép đo nào cho bạn quyền tiêu cả ba tháng vào đúng một nỗi đau, mà không cần đọc thêm một paper nào?

1.1 Đặt vấn đề

Bạn đang bảo trì một nhánh ORB-SLAM3 stereo-inertial [1]. Nó chạy EuRoC [2], TUM-VI [3] và OpenLORIS, triển khai trên Jetson Orin, và đã được tinh chỉnh nhiều tháng. Trên phần lớn chuỗi EuRoC nó chạy trơn chuỗi và cho một quỹ đạo mà bạn đã đo đi đo lại nhiều lần, nên bạn biết rất rõ mức sai số của nó — biết bằng số đo của chính mình chứ không bằng bảng của ai. Mỗi tuần lại có một paper nói rằng một mạng nào đó làm tốt hơn.

Câu hỏi thật không phải “học sâu có tốt hơn hình học không”. Câu hỏi đó vô nghĩa vì không có phép đo nào trả lời được nó. Câu hỏi thật gồm bốn phần, và phải hỏi theo đúng thứ tự: khối nào trong hệ của tôi đang thật sự làm hỏng việc, trên chuỗi nào, với ngân sách bao nhiêu mili-giây, và khi nó hỏng thì tôi có biết không.

Có một sự bất đối xứng khiến việc này khó hơn vẻ ngoài. Paper được viết để cho thấy nơi phương pháp *thắng*. Bạn cần biết nơi nó *thua*. Không phải vì tác giả không trung thực, mà vì cấu trúc khuyến khích của việc công bố không thưởng cho một bảng liệt kê các cảnh mà phương pháp gây. Hệ quả là bạn phải tự dựng phần thiếu đó.

Còn một bất đối xứng nữa, nặng hơn. Baseline trong phần lớn paper là ORB-SLAM3 chạy cấu hình mặc định. Baseline của bạn là ORB-SLAM3 đã được chính bạn tinh chỉnh cho cảm biến của mình. Khoảng cách giữa hai thứ đó không nhỏ. Một phần đáng kể của “cải thiện nhiều lần” trong bảng có thể chỉ là khoảng cách tinh chỉnh, chứ không phải đóng góp của phương pháp.

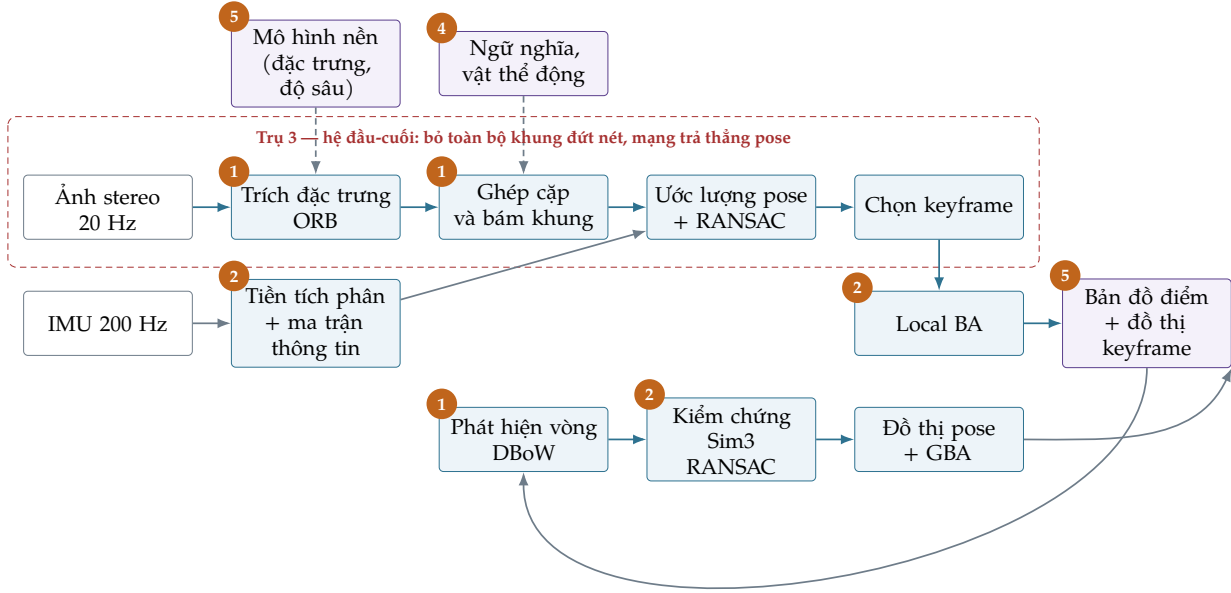
↪ **tiền đề** E/24 — đây chính là ràng buộc “baseline mạnh là nghĩa vụ” của chương 24, lần này nhìn từ phía người đọc bằng chứ không phải người làm bảng

Chương này vì thế là một bộ lọc, không phải một khảo sát. Nó có bốn phần và bốn phần đó dùng được độc lập. Phần đầu nói *có thể* cắm mạng vào đâu, tức năm trụ. Phần hai nói *có nên* cắm không, tức khung bốn cổng. Phần ba nói bốn nỗi đau cụ thể của bạn sang nhóm phương pháp nào hứa chữa và chương nào nói về nó. Phần bốn nói *cắm sâu tới mức nào*, tức ba mức can thiệp. Phần cuối là kỹ năng thao tác: đọc một paper trong ba mươi phút và biết loại nó lúc nào.

Một lời cảnh báo về giọng. Chương này sẽ kết luận “chưa đáng” nhiều lần. Đó không phải thái độ bảo thủ mà là hệ quả trực tiếp của việc bạn đã có một hệ thống chạy tốt. Với người chưa có hệ thống nào, một hệ đầu-cuối học được là con đường nhanh nhất để có thứ chạy được. Với người đã có, phần lớn phương pháp trong Phần F là một cuộc đổi chác đất: đổi tính chẩn đoán được, tính lặp lại và ngân sách tính toán, lấy một cải thiện chỉ xuất hiện trên vài cảnh.

1.2 Năm trụ: học sâu cắm được vào đâu (The Five Pillars)

Trước khi hỏi “có nên”, phải biết “ở đâu”. Một hệ SLAM cổ điển không phải một khối. Nó là một chuỗi khối nối nhau bằng những giao diện rất rõ ràng, và chính các giao diện đó mới là chỗ một mạng có thể chen vào. Hình 1.1 vẽ hệ thống đó cùng năm chỗ cắm.



Hình 1.1. Một hệ stereo-inertial cổ điển và năm chỗ có thể cắm mạng vào. Trụ 1 đổi cách *sinh ra* phép đo (đặc trưng, ghép cặp, lấy ứng viên vòng). Trụ 2 đổi cách *cân* phép đo trong bộ tối ưu. Trụ 3 xóa cả khung đứt nét đồ và thay bằng một mạng trả thẳng pose. Trụ 4 đổi *phép đo nào được phép vào*. Trụ 5 đổi *bản đồ là cái gì*, hoặc cấp một tiên nghiệm học sẵn cho các khối khác. Đi từ 1 tới 3 là đi từ ngoài vào trong: càng vào sâu, càng ít đại lượng trung gian mà bạn còn kiểm tra được bằng một kỳ vọng vật lý.

A. Trụ 1 – frontend nhận thức

Trụ này thay cách *sinh ra phép đo*. ORB là một bộ dò góc cộng một chuỗi bit so sánh cường độ điểm ảnh. Thay nó bằng một mạng nghĩa là: ảnh vào, danh sách điểm ảnh cùng vectơ mô tả ra. Ghép cặp cũng vậy, và tầng lấy ứng viên vòng cũng vậy. SuperPoint [4] là ví dụ điển hình cho bộ dò và mô tả; SuperGlue [5] và LightGlue [6] cho bước ghép cặp; LoFTR [7] bỏ hẳn bước dò điểm.

Điều quan trọng về mặt kiến trúc là giao diện không đổi. Backend vẫn nhận đúng thứ nó vẫn nhận: các tương ứng hai chiều kèm một ước lượng độ bất định. Vì thế RANSAC, hiệu chỉnh chùm tia, đồ thị nhân tử, tất cả giữ nguyên. Đây là trụ có rủi ro thấp nhất và cũng là trụ đã chín nhất, và chương F/28 dành trọn cho nó.

B. Trụ 2 – backend tối ưu

Trụ này không đổi phép đo, nó đổi *trọng số* của phép đo. Trong một đồ thị nhân tử, mỗi cạnh có một ma trận thông tin. Với IMU, ma trận đó đến từ mô hình nhiễu của cảm biến qua bước tiên tích phân [8]. Với điểm ảnh, nó thường là một hằng số nhân với hệ số tầng kim tự tháp. Cả hai đều là giả định, và cả hai đều sai ở mức nào đó.

Học sâu vào đây theo ba đường. Thứ nhất, học độ bất định của từng phép đo thay vì dùng hằng số. Thứ hai, học mô hình nhiễu của IMU, gồm cả trôi bias và nhiễu phụ thuộc chuyển động. Thứ ba, làm cho cả bộ tối ưu khả vi để huấn luyện xuyên qua nó. Hai đường đầu nằm ở chương F/34,

đường thứ ba ở chương F/31. Đây là trụ ít được khai thác nhất và, theo đánh giá của chương này, là trụ có tỉ số lợi ích trên rủi ro tốt nhất cho một hệ đã chạy.

C. Trụ 3 — hệ đầu-cuối

Trụ này xoá cả chuỗi. Ảnh vào, pose ra, không có bước nào ở giữa mà bạn đặt tên được bằng một khái niệm hình học. DROID-SLAM [9] là mốc quan trọng nhất của dòng này; nó vẫn giữ một lối tối ưu nhưng toàn bộ phần sinh tương ứng là học được. Các hệ mới hơn đi xa hơn nữa, đưa cả bước dựng hình vào một mô hình nền duy nhất.

Cái được là tính bền: những hệ này thường không mất bám ở chỗ mà hệ đặc trưng thừa mất bám. Cái mất là toàn bộ bề mặt chẩn đoán. Không còn số inlier để nhìn, không còn hiệp phương sai để hỏi, không còn cách nào trả lời câu “vì sao khung 1842 sai”. Với một hệ sản phẩm, đó là cái giá rất khó trả. Chương F/31 nói kỹ.

D. Trụ 4 — hiểu cảnh

Trụ này không ước lượng pose và không sinh phép đo. Nó quyết định *phép đo nào được phép vào*. Ví dụ điển hình là loại bỏ điểm nằm trên vật thể chuyển động: DynaSLAM [10] dùng mặt nạ ngữ nghĩa để vớt các điểm rơi trên người và xe trước khi chúng vào bộ tối ưu. Ở mức cao hơn, đồ thị cảnh 3D như Kimera [11] dựng một tầng ngữ nghĩa nằm *cạnh* bộ ước lượng hình học chứ không thay nó.

Đặc điểm cấu trúc của trụ này rất đáng chú ý: nó chỉ *trừ đi* chứ không cộng thêm. Nếu mạng chết, hệ thống trở về đúng hành vi cổ điển. Đó là lý do nó thuộc mức can thiệp thấp nhất, dù mô hình bên trong có thể rất lớn. Chương F/33 nói kỹ.

E. Trụ 5 — mô hình nền và biểu diễn neural

Trụ này gồm hai thứ hay bị gộp làm một, và tách chúng ra là việc đầu tiên phải làm.

- **Mô hình nền hình học** — một mạng lớn huấn luyện trên rất nhiều dữ liệu, trả về độ sâu, đặc trưng, hoặc trực tiếp một đám mây điểm đã căn chỉnh. DUST3R [12] là mốc; MAST3R-SLAM [13] là một nỗ lực đưa dòng đó vào SLAM thời gian thực. Chúng cấp *tiền nghiệm* cho các khối khác.
- **Biểu diễn bản đồ neural** — NeRF [14] và 3D Gaussian Splatting [15] đổi câu trả lời cho câu hỏi “bản đồ là cái gì”. Thay vì một đám mây điểm thưa, bản đồ thành một trường liên tục hoặc một tập splat. MonoGS [16] là ví dụ đưa nó vào một vòng SLAM.

Hai thứ đó phục vụ hai mục đích khác nhau. Mô hình nền nhắm vào độ bền của phép đo. Biểu diễn neural nhắm vào chất lượng bản đồ dựng lại, mà chất lượng đó *không* đồng nghĩa với độ chính xác quỹ đạo. Trộn hai mục tiêu là nguồn hiểu nhầm lớn nhất khi đọc mẩu này, và chương F/32 mở đầu bằng đúng việc tách chúng.

Bảng 1.1. Năm trụ đọc theo cùng bốn tiêu chí. Cột cuối là cột đáng giá nhất: nó cho biết khi nào lợi ích hứa hẹn của trụ đó biến mất.

Trụ	Cơ chế	Giả định	Hỏng khi nào
1. Frontend nhận thức	Mạng sinh điểm, mô tả, tương ứng, hoặc ứng viên vòng; giao diện với backend không đổi	Cảnh triển khai nằm trong phân phối huấn luyện; có đủ ngân sách độ trễ	Cảnh lạ về kết cấu hoặc ánh sáng; mạng vẫn trả đủ điểm nhưng mô tả không còn phân biệt được
2. Backend tối ưu	Học trọng số, hiệp phương sai, hoặc mô hình nhiễu IMU; cấu trúc đồ thị nhân tử giữ nguyên	Độ bất định thật sự phụ thuộc dữ liệu quan sát được, không chỉ phụ thuộc cảm biến	Trọng số học được quá tự tin; bộ tối ưu tin nhầm một cạnh sai và không có tín hiệu nào báo
3. Hệ đầu-cuối	Mạng nhận ảnh và trả pose hoặc trả trực tiếp quỹ đạo; xoá các bước hình học tường minh	Chuyển động và cảnh khi triển khai giống lúc huấn luyện; tài nguyên GPU dồi dào	Chuỗi dài hơn nhiều so với lúc huấn luyện; bộ nhớ tăng theo số khung; không còn đại lượng trung gian để chẩn đoán
4. Hiểu cảnh	Mạng quyết định phép đo nào bị loại, hoặc dựng một tầng ngữ nghĩa cảnh bản đồ hình học	Vật thể động nhận dạng được bằng loại của nó; loại bỏ nhầm ít hơn giữ lại nhầm	Vật động không thuộc lớp đã học (rèm cửa, bóng, phản chiếu); cảnh mà phần lớn điểm nằm trên vật động
5. Mô hình nền, bản đồ neural	Tiên nghiệm học sẵn cho độ sâu và đặc trưng, hoặc đổi hẳn cấu trúc bản đồ	Tiên nghiệm đủ đúng để không kéo lệch hình học; đủ bộ nhớ giữ biểu diễn	Sai tỉ lệ tích lũy; chuỗi dài ngoài trời; ảnh dựng lại đẹp trong khi quỹ đạo tệ đi

F. Vì sao thứ tự năm trụ lại chính là thứ tự lịch sử

Năm trụ không phải một danh sách phẳng. Chúng xếp theo độ sâu của can thiệp, và độ sâu đó gần như trùng với thứ tự mà cộng đồng đã đi qua. Mạch dưới đây giải thích vì sao.

I Mạch 1 — vì sao học sâu vào SLAM từ frontend trước

- Vấn đề** Hệ cổ điển đo cảnh bằng đặc trưng do người thiết kế. ORB rẻ, bất biến với xoay trong mặt phẳng ảnh, và chạy được trên CPU. Nhưng nó giả định ảnh sắc nét, có vân, và độ sáng ổn định.

Giải pháp Thay bộ dò và bộ mô tả bằng một mạng, huấn luyện trên các cặp ảnh đã biết tương ứng.

Giả định Cảnh triển khai giống cảnh huấn luyện, và có GPU rảnh để chạy suy luận mỗi khung.

Giá phải trả thêm độ trễ, thêm bộ nhớ, thêm một phụ thuộc phần cứng.
- Vấn đề** Đầu ra vẫn là điểm và mô tả, nên backend không phải sửa dòng nào. Đó chính là điều làm bước này hấp dẫn: rủi ro tích hợp thấp.

Giải pháp Giữ nguyên toàn bộ phần hình học, chỉ đổi nguồn phép đo.

Vấn đề mới Bạn mất một tín hiệu cũ rất quý. ORB không tìm được góc nào thì trả về danh sách rỗng, và hệ thống biết ngay là đang mù. Một mạng gần như luôn trả đủ số điểm bạn yêu cầu, kể cả trên tường trắng.
- Vấn đề** Nếu phép đo đã học được, vì sao còn cân chúng bằng hằng số do người đặt? Ma trận thông tin của cảnh ảnh trong phần lớn hệ thống là một hằng số nhân hệ số tăng.

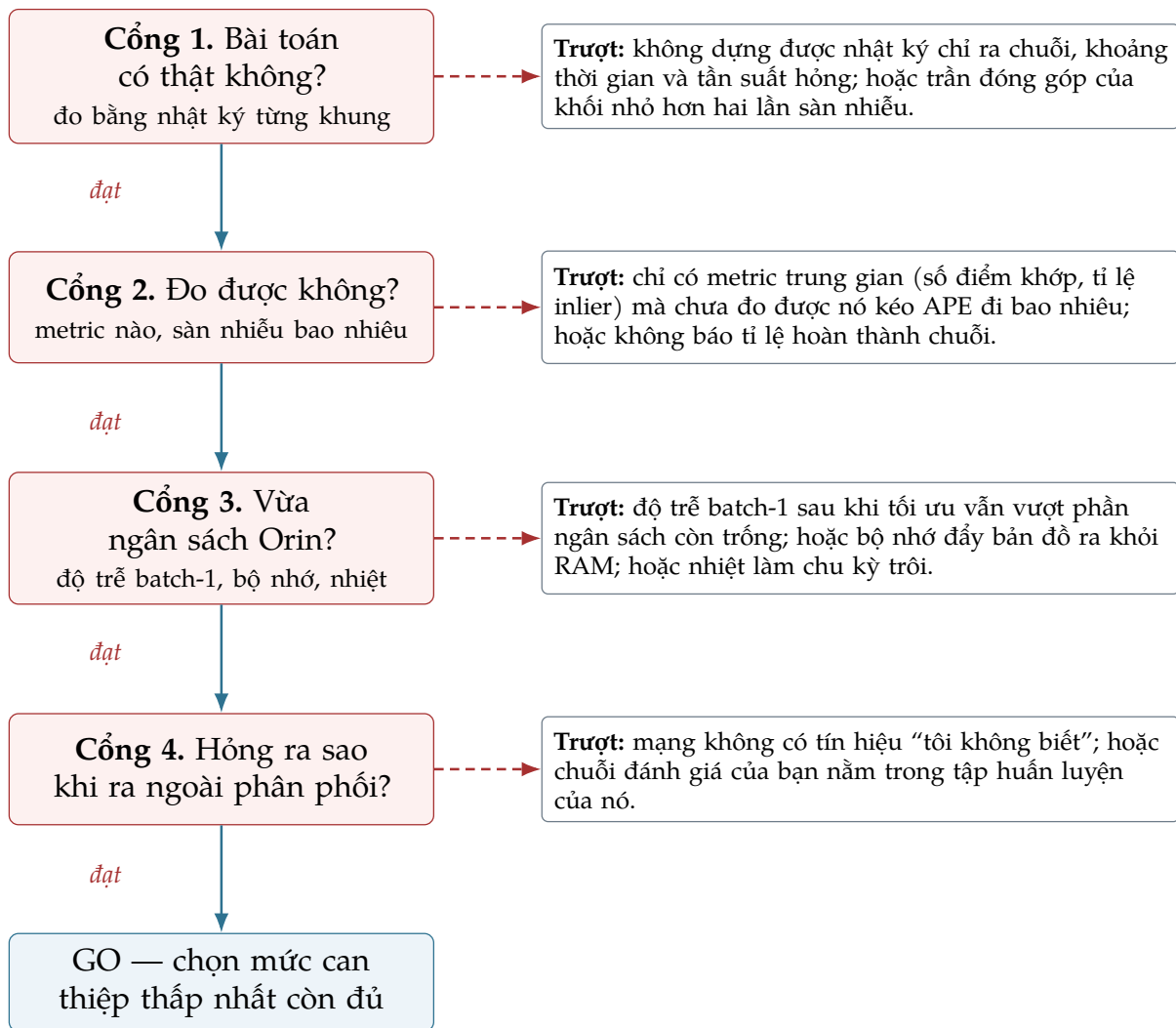
Giải pháp Học độ bất định của từng phép đo, hoặc học mô hình nhiễu IMU.

Giải định	Có chân trị đủ tốt để dạy mạng biết khi nào phép đo của chính nó sai.
Hệ quả	Đây là trụ 2. Nó khó hơn trụ 1 vì nhãn khó kiểm, nhưng nó nằm hoàn toàn bên trong bộ ước lượng cổ điển, nên không phá vỡ gì.
4 Vấn đề	Cả hai bước trên vẫn bị chặn bởi thiết kế của hệ cổ điển: đặc trưng thưa, tương ứng rời rạc, keyframe rời rạc.
Giải pháp	Làm cho cả chuỗi khả vi rồi huấn luyện đầu-cuối.
Giá phải trả	mất bề mặt chẩn đoán, mất khả năng quay lại, và mất luôn nhiều năm tính chỉnh đã nằm trong các ngưỡng của hệ cũ.
Vấn đề mới	Khi hệ đầu-cuối sai, bạn không còn đại lượng trung gian nào để hỏi. Cách sửa duy nhất là huấn luyện lại, và đó là vòng lặp tính bằng tuần chứ không bằng phút.

Mạch này cho một quy tắc đọc rất hữu dụng. Khi gặp một phương pháp mới, hãy hỏi ngay: nó đang ở trụ nào? Câu trả lời cho biết bạn phải xoá cái gì trong hệ hiện tại để nhận nó, và đó thường là thông tin quyết định, chứ không phải con số APE trong bảng.

1.3 Khung bốn cổng (The Four-Gate Decision Framework)

Biết một phương pháp nằm ở trụ nào chưa đủ để quyết định. Cần một quy trình loại bỏ, và quy trình đó phải rẻ. Bốn cổng dưới đây được xếp theo chi phí tăng dần: cổng 1 tốn vài ngày đo trên hệ của bạn, cổng 4 tốn nhiều nhất và vì thế đứng cuối. Hình 1.2 là sơ đồ đầy đủ, kèm lối ra ở mỗi cổng.



Hình 1.2. Bốn cổng xếp theo chi phí tăng dần, mỗi cổng có tiêu chí trượt cụ thể. Phần lớn ứng viên chết ở cổng 1 và cổng 3, và cả hai cổng đó đều trả lời được bằng phép đo trên hệ của chính bạn, không cần chạy phương pháp mới lần nào.

A. Cổng 1 – bài toán có thật không, và đo bằng gì

Đây là cổng loại được nhiều ứng viên nhất, và cũng là cổng hay bị bỏ qua nhất. Lý do bỏ qua rất người: một paper mô tả rất sống động một thất bại, bạn nhận ra hệ mình cũng có thất bại đó, và thế là “vấn đề đã được xác nhận”. Nhưng nhận ra không phải là đo.

Cổng này đòi hai thứ. Thứ nhất là **nhật ký từng khung**: chuỗi nào, khoảng thời gian nào, triệu chứng gì, tần suất bao nhiêu. Nếu bạn nói “hệ hay mất bám khi quay nhanh” mà không nói được nó xảy ra ở bao nhiêu khung trên tổng số bao nhiêu, bạn chưa qua cổng. Thứ hai là **trần đóng góp**: thay khối bị nghi bằng chân trị rồi đo lại cả hệ.

Bước thứ hai là bước quyết định, và nó rõ hơn về ngoài. Muốn biết trần của một bộ ghép cặp tốt hơn, không cần bộ ghép cặp nào. Dùng pose chân trị để chiếu ngược và loại mọi tương ứng sai, rồi chạy lại. Con số thu được là mức tốt nhất mà *bất kỳ* bộ ghép cặp nào cũng không vượt qua được. Gọi nó là c : mọi paper hứa hẹn về ghép cặp đều bị chặn trên bởi đúng c , bất kể bảng của họ đẹp tới đâu. Và c chỉ có nghĩa khi nó là số bạn tự đo trên chuỗi của mình.

↪ **trường hợp riêng của E/24** — oracle experiment ở chương 24, áp cho một hệ SLAM: nó biến câu hỏi “nên đầu tư vào đâu” thành một phép đo chạy trong một ngày

Có một điều kiện trượt thứ ba, ít trực giác hơn nhưng rất hay xảy ra: thất bại giải thích được bằng một lỗi cấu hình. Trước khi kết luận rằng frontend yếu, hãy kiểm tra ngưỡng, số tầng kim tự tháp, chính sách chọn keyframe, và thời gian “vừa mới định vị lại”. Sửa những thứ đó không tốn gì cả, và chúng giải thích một phần lớn các thất bại nghe có vẻ cần học sâu.

B. Cổng 2 — đo được không, metric nào, sần nhiều bao nhiêu

Giả sử vấn đề có thật. Câu tiếp theo: bạn sẽ nhận ra cải thiện bằng cách nào? Cổng này có ba yêu cầu.

- **Metric đầu cuối, không phải metric trung gian.** Số điểm khớp, tỉ lệ inlier, độ chính xác ghép cặp đều là đại lượng trung gian. Chúng có thể tăng mà APE không nhúc nhích, vì bộ tối ưu vốn đã loại được phần lớn phép đo xấu. Nếu bạn định dùng metric trung gian thì phải đo trước mỗi liên hệ của nó với APE trên chính hệ của bạn.
- **Tỉ lệ hoàn thành chuỗi báo cùng APE.** Đây là cái bẫy đặc thù của SLAM. Một hệ mất bám giữa chừng chỉ được chấm trên đoạn nó còn sống, mà đoạn đầu của một chuỗi hầu như luôn là phần dễ. APE của nó vì thế đẹp một cách vô nghĩa, và càng mất bám sớm thì trông càng đẹp.
- **Sần nhiều đo trước.** Nhánh của bạn phi tất định: thứ tự duyệt các container đánh chỉ mục bằng con trỏ và tranh chấp giữa luồng bám và luồng dựng bản đồ đều làm kết quả đổi giữa các lần chạy. Vì thế mọi so sánh phải chạy nhiều lần, và “nhiều” là một con số tính ra được từ độ lệch chuẩn giữa các lần chạy.

Tiêu chí trượt của cổng này rất gọn: nếu mức cải thiện mà phương pháp hứa nhỏ hơn sần nhiều bạn đã đo, thì bạn không có cách nào xác nhận nó, và việc tích hợp là một canh bạc chứ không phải một quyết định kỹ thuật.

Ví dụ tính tay, chạy trong đầu được. Mọi con số dưới đây là số *giả định* để thấy hình dạng của công thức, không phải số đo của một hệ nào. Giả sử bạn đo sần nhiều và được $\sigma = 0,6$ cm APE giữa các lần chạy. Bạn muốn phát hiện một cải thiện thật $\Delta = 1,0$ cm. Công thức cỡ mẫu quen thuộc cho số lần chạy mỗi cấu hình:

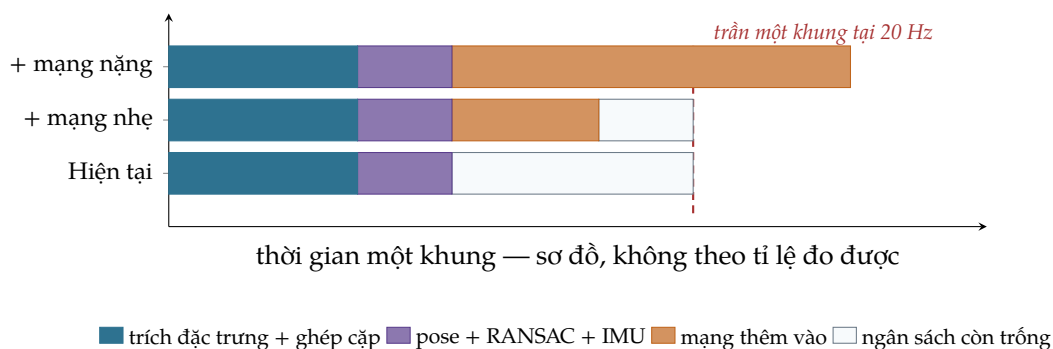
$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\Delta^2} = \frac{2(1,96 + 0,84)^2 (0,6)^2}{(1,0)^2} \approx 5,6.$$

Tức khoảng sáu lần chạy mỗi bên, mười hai lần cho một so sánh. Bây giờ thử $\Delta = 0,4$ cm: mẫu số nhỏ đi hơn sáu lần, nên n vọt lên khoảng 35 mỗi bên. Hình dạng đó nói thẳng một điều rất hữu ích, và nó đúng bất kể σ của bạn bằng bao nhiêu: số lần chạy tăng theo *bình phương* tỉ số giữa sần nhiều và mức cải thiện cần phát hiện. Nghĩa là có một ranh giới cứng — cải thiện nhỏ hơn sần nhiều chừng một nửa thì nằm ngoài tầm ngân sách thí nghiệm của bạn, và một paper hứa hẹn ở mức đó đã trượt cổng 2 trước cả khi bạn tải mã nguồn về. Ranh giới cứng đó nằm ở đâu thì bạn thay σ đo được của mình vào công thức trên là ra.

C. Cổng 3 — có vừa ngân sách Jetson Orin không

Cổng này là số học, và chính vì là số học nên nó loại được rất nhanh. Bắt đầu từ tốc độ khung hình. Camera EuRoC chạy 20 Hz, tức 50 mili-giây mỗi khung [2]. Đó là toàn bộ ngân sách của luồng bám, và ngân sách đó đã có người ở: trích đặc trưng trên hai ảnh, ghép cặp, RANSAC, tối ưu pose. Phần còn trống là phần bạn được tiêu.

Hình 1.3 vẽ *hình dạng* của phép tính đó và cố ý không mang một con số nào. Nó chỉ phát biểu ba điều: ngân sách là một cái hộp có kích thước cố định, cái hộp đó đã có người ở, và một mạng thêm vào thì hoặc lọt vào phần trống hoặc đẩy cả luồng bám vượt trần. Việc đầu tiên khi qua cổng này là điền số đo thật của chính bạn vào từng dải.



Hình 1.3. Hình dạng của phép tính ngân sách ở cổng 3. Trục không có vạch số và các dải không phải số đo của bất kỳ máy nào — chúng chỉ nói rằng cái hộp có kích thước cố định và đã có người ở. Ba điều đáng mang đi là: một mạng đủ nhẹ thì lọt vào phần còn trống; một mạng nặng thì đẩy luồng bám vượt trần và hệ bắt đầu bỏ khung; và phần còn trống là đại lượng bạn phải *đo* trên Orin, ở đúng chế độ điện và sau khi máy đã nóng, trước khi nói bất cứ điều gì về ứng viên.

Bốn cái bẫy làm phép tính này sai lệch, và cả bốn đều đẩy về hướng lạc quan.

- **Số FPS trong paper là số thông lượng.** Nó đo với batch lớn, ảnh đã nằm sẵn trên GPU, và thường không tính khâu tiền xử lý. SLAM chạy batch bằng 1 và phải trả cả chi phí chuyển dữ liệu. Chênh lệch giữa hai chế độ này không phải chuyện xê dịch vài phần trăm: nó lớn tới mức một kết luận rút ra từ FPS trong bảng hoàn toàn có thể đảo chiều khi bạn tự đo.
- **GPU của bạn có thể đã bận.** Nếu bạn đã chuyển trích đặc trưng sang GPU, thì mạng mới không chạy trên một GPU rảnh mà tranh chấp với chính phần đó.
- **Bộ nhớ dùng chung.** Trên Orin, CPU và GPU chia nhau cùng vùng nhớ. Một mô hình chiếm vài gigabyte không chỉ tốn bộ nhớ của nó, nó lấy đi chỗ của bản đồ.
- **Nhiệt.** Số đo trong ba mươi giây đầu không phải số đo lúc vận hành. Chu kỳ trôi khi máy nóng, và một hệ vừa đủ 50 ms lúc nguội sẽ bỏ khung sau mười phút.

Có một phép sàng nhanh giúp loại ứng viên *trước* khi cài đặt bất cứ thứ gì, và nó chỉ cần hai đại lượng. Tử số: số phép nhân cộng mà mạng cần cho một ảnh — đọc từ bảng của paper, hoặc đếm bằng công cụ. Mẫu số: thông lượng mà máy của bạn *thật sự* đạt được ở batch bằng 1, ở độ chính xác bạn định dùng, trên đúng dạng phép toán đó. Chia tử cho mẫu ra một khoảng thời gian, rồi nhân đôi nếu là ảnh stereo.

Toàn bộ chỗ dễ sai nằm ở mẫu số, và nó sai theo một hướng cố định là hướng lạc quan. Hiệu năng đỉnh ghi trên tờ thông số là con số đạt được ở batch lớn với ma trận lớn; ở batch bằng 1, phần lớn thời gian trôi vào việc chờ dữ liệu chứ không vào việc nhân, và với các phép toán ma trận nhỏ thì thứ chặn là băng thông bộ nhớ chứ không phải số nhân. Vì thế mẫu số phải là một con số bạn đã *đo* bằng một mạng nhỏ đã biết trên chính máy đó, không phải một con số bạn tra được.

Giá trị của phép chia này không nằm ở kết quả mà ở bậc độ lớn của nó, và nó cho ba phán quyết chứ không cho một con số. Nếu kết quả lớn hơn phần ngân sách còn trống nhiều lần thì trượt, khỏi cần cài đặt. Nếu nó cùng bậc với phần còn trống thì chưa kết luận được gì, và đáng bỏ một ngày ra đo thật. Nếu nó nhỏ hơn hẳn thì phép chia này đã hết việc, và rủi ro chuyển sang ba cái bẫy còn lại ở trên: tranh chấp GPU, bộ nhớ dùng chung, và nhiệt.

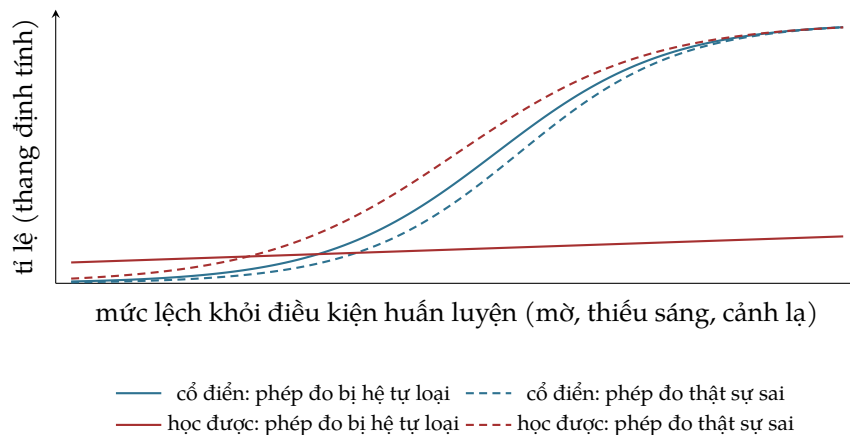
↔ **ràng buộc bởi B/11** — đây chính là chỗ “bộ nhớ và băng thông, không phải FLOPs, mới là thứ chặn” của chương 11, xuất hiện dưới dạng một ngân sách mili-giây cứng

D. Cổng 4 – hỏng ra sao khi ra ngoài phân phối

Ba cổng đầu nói về lợi ích và chi phí. Cổng này nói về rủi ro, và với một hệ SLAM nó quan trọng hơn cả ba cổng kia cộng lại.

Lý do nằm ở cấu trúc, không nằm ở độ chính xác. SLAM là một vòng phản hồi. Phép đo đi vào bản đồ, bản đồ quyết định phép đo tiếp theo được ghép với cái gì. Một phép đo sai không chỉ làm sai một khung; nó ở lại trong bản đồ. Với một vòng đóng sai, hậu quả gần như không sửa được: cả một đoạn quỹ đạo bị kéo lệch vĩnh viễn.

Bây giờ so sánh hai kiểu hỏng. Khi ORB gặp một bức tường trắng, nó không tìm được góc nào. Số điểm về gần 0, RANSAC thất bại, hệ thống tuyên bố mất bám và chuyển sang định vị lại. Thất bại đó *ồn ào*: nó tự báo cáo. Khi một mạng gặp bức tường trắng, nó vẫn trả về đúng số điểm bạn yêu cầu, kèm điểm tin cậy trông bình thường. Thất bại đó *im lặng*, và im lặng là chỗ chết người trong một vòng phản hồi.



Hình 1.4. Hai kiểu hỏng, vẽ theo dạng định tính chứ không phải theo số đo. Với khối cổ điển, hai đường gần như trùng nhau: cái gì sai thì cũng bị chính hệ thống loại, nên bạn nhìn số inlier là biết. Với khối học được, hai đường tách ra: tỉ lệ sai tăng trong khi hệ thống vẫn nhận vào bình thường. Khoảng cách giữa hai đường đồ chính là lượng phép đo sai chảy thẳng vào bản đồ mà không ai chặn.

TRỰC GIÁC

Hãy nghĩ về bộ tối ưu như một quan toà và các phép đo như nhân chứng. Ma trận thông tin là mức tin cậy toà dành cho từng nhân chứng, và RANSAC là phần đối chất. Một bộ dò đặc trưng cổ điển là nhân chứng thật thà kiểu cũ: hỏi tới chỗ không thấy thì nói thẳng “tôi không biết”. Một mạng chưa được hiệu chỉnh là nhân chứng ăn nói lưu loát, không bao giờ nói “tôi không biết”, và trả lời tự tin cả về những việc chưa từng chứng kiến. Vấn đề không phải nhân chứng ấy nói sai nhiều hơn. Vấn đề là toà mất khả năng biết lúc nào nên giảm mức tin cậy, vì tín hiệu tự tin đã không còn mang thông tin.

Từ đó ra hai tiêu chí trượt của cổng 4. Thứ nhất: nếu mạng không có một tín hiệu “tôi không biết” dùng được trong lúc chạy, thì nó chỉ nên được cấm ở nơi mà đầu ra của nó *trừ đi* chứ không cộng vào, hoặc phải có một tầng kiểm chứng hình học đứng sau. Thứ hai: nếu chuỗi bạn dùng để đánh giá nằm trong tập huấn luyện của mô hình, thì con số bạn đo được không nói gì về hành vi khi triển khai. Điều này xảy ra thường xuyên hơn người ta tưởng, vì EuRoC và TUM-VI là các tập rất phổ biến.

→ **cùng nguyên lý với B/08** — đây là dịch chuyển phân phối nhìn từ phía hệ thống: cái đặt không phải độ chính xác giảm, mà là mất tín hiệu báo rằng độ chính xác đã giảm

Bảng 1.2. Bốn cổng, phép đo quyết định của từng cổng, và tiêu chí trượt. Cột thứ ba là chỗ nên đầu tư công sức: nó nói bạn phải chạy thí nghiệm nào, chứ không phải đọc thêm paper nào.

Cổng	Câu hỏi	Phép đo quyết định	Tiêu chí trượt
1	Bài toán có thật không	Nhật ký từng khung kèm tần suất; oracle thay khối bằng chân trị để lấy trần đóng góp	Không dựng được nhật ký; trần nhỏ hơn hai lần sàn nhiễu; thất bại giải thích được bằng một tham số cấu hình
2	Đo được không	Sàn nhiễu từ nhiều lần chạy cùng cấu hình; APE kèm tỉ lệ hoàn thành; kiểm mối liên hệ giữa metric trung gian và APE	Cải thiện hứa hẹn nhỏ hơn sàn nhiễu; chỉ có metric trung gian; không báo tỉ lệ hoàn thành
3	Vừa ngân sách Orin không	Hồ sơ thời gian luồng bám trên Orin; độ trễ batch-1 của mạng sau khi tối ưu, đo lúc máy đã nóng	Vượt phần ngân sách còn trống; bộ nhớ lẫn bản đồ; chu kỳ trôi theo nhiệt
4	Hông ra sao khi ra ngoài phân phối	Chạy trên cảnh cố ý khác: thiếu sáng, mờ, hành lang không vắn; đo tỉ lệ phép đo sai lọt qua tầng kiểm chứng	Không có tín hiệu “tôi không biết”; chuỗi đánh giá nằm trong tập huấn luyện; hông im lặng ở chỗ cộng thêm phép đo

E. Chạy thử khung: một ứng viên đi qua bốn cổng

Khung chỉ có nghĩa khi chạy được trên một ví dụ. Lấy ứng viên phổ biến nhất: thay ORB bằng một bộ dò và mô tả học được, kèm một bộ ghép cặp học được. Về vị trí, nó là trụ 1 ở mức can thiệp 2. Dưới đây cố ý không có một con số nào, vì con số phải là của bạn. Thứ đáng học ở ví dụ này là *thứ tự câu hỏi* và *dạng kết luận* mà mỗi cổng cho phép phát biểu.

Cổng 1. Bạn dựng nhật ký và thấy sai số không rải đều mà dồn vào các đoạn xoay nhanh. Bạn chạy oracle ghép cặp: dùng pose chân trị loại mọi tương ứng sai rồi chạy lại. Kết luận có đúng một dạng, và dạng đó là một tỉ số: trần đóng góp lớn hơn sàn nhiễu *bao nhiêu lần*. Lớn hơn nhiều lần thì qua cổng. Xấp xỉ bằng thì dừng lại ngay ở đây, và bạn vừa tiết kiệm toàn bộ phần còn lại.

Cổng 2. Bạn báo APE kèm tỉ lệ hoàn thành, và bạn đã có sàn nhiễu từ nhiều lần chạy cùng một cấu hình. Bạn cũng ghi chú rằng không được kết luận bằng số điểm khớp: đó là metric trung gian, và cái trần đo được ở cổng 1 mới là thứ đáng so. Qua cổng.

Cổng 3. Đây là chỗ ứng viên gặp khó, và lý do là cấu trúc chứ không phải một con số. Bộ dò chạy trên hai ảnh mỗi khung, bộ ghép cặp chạy thêm một lần nữa mỗi khung, và cả ba khoản đó nằm trong luồng bám, tức là cùng chen vào một ngân sách vốn đã có người ở. Với một mô hình cỡ trung, kết luận thường là trượt — nhưng trượt *có điều kiện*, và điều kiện đó rất cụ thể.

Ở đây có một lối thoát mà phép tính thô che mất. Không phải mọi chỗ dùng ghép cặp đều chạy ở nhịp camera. Tầng ghép cặp trong đường đóng vòng chỉ chạy khi có ứng viên, tức vài lần mỗi phút, nên ngân sách của nó rộng hơn ngân sách một khung của luồng bám nhiều bậc. Vậy cùng một mô hình trượt cổng 3 ở luồng bám lại lọt thoải mái ở đường đóng vòng. Đó là kết luận đáng giá nhất của cả ví dụ này, và nó chỉ hiện ra khi bạn tính ngân sách *theo từng đường*, không tính theo cả hệ.

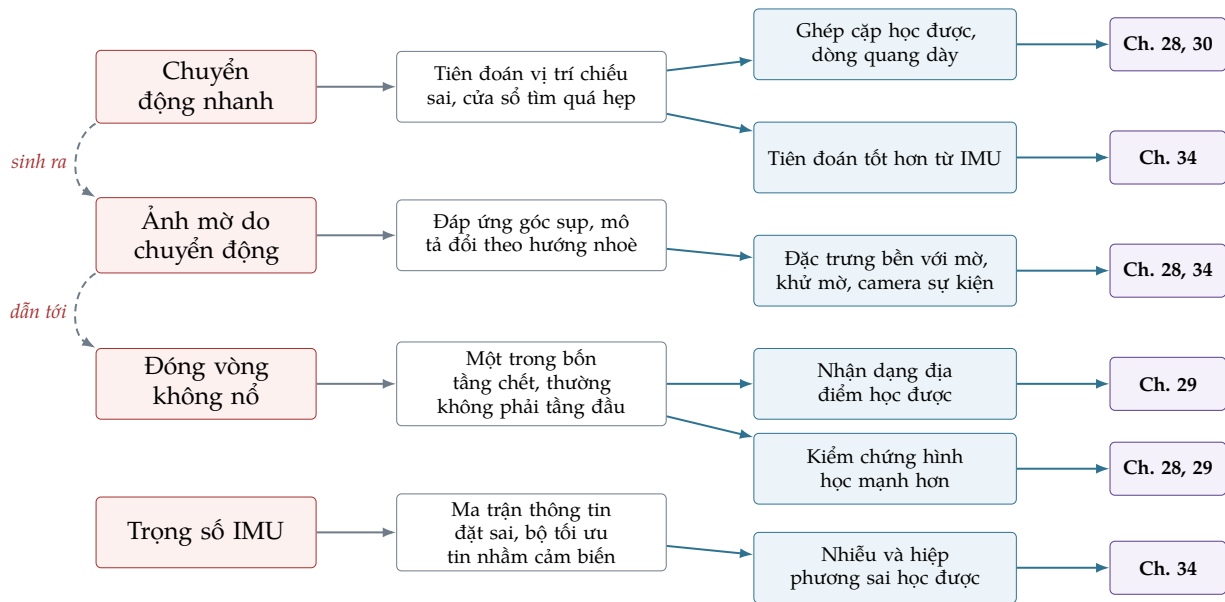
Cổng 4. Bộ dò học được không có tín hiệu “tôi không biết” dùng được. Nhưng ở đường đóng vòng, đằng sau nó vẫn còn tầng kiểm chứng Sim3 bằng RANSAC, tức vẫn còn một trọng tài hình học. Rủi ro vì thế bị chặn: một tương ứng sai muốn vào bản đồ phải qua được kiểm chứng hình học. Điều kiện kèm theo là bạn không được nới ngưỡng inlier để bù cho việc mô hình mới trả nhiều tương ứng hơn.

Phán quyết. Không phải một chữ GO hay NO-GO, mà là một câu có điều kiện: chưa đưa vào luồng bám vì cổng 3, nhưng đáng thử ở đường đóng vòng vì ở đó cổng 3 rộng và cổng 4 đã có người canh. Chi phí thử: cỡ một tuần. Điều kiện xem lại phán quyết đầu tiên: khi độ trễ của luồng bám giảm đủ để mở ra một chỗ trống thật, hoặc khi có một mô hình nhẹ hơn ở cùng chất lượng.

1.4 Bản đồ bốn nỗi đau (Mapping Four Pains to Method Families)

Đây là mục trung tâm của chương. Bốn công cho bạn cách loại; mục này cho bạn cách *chọn*. Nó trả lời một câu duy nhất: với mỗi nỗi đau cụ thể, nhóm phương pháp nào hứa chữa, chương nào nói về nó, và điều kiện gì phải đúng thì mới đáng thử.

Trước khi vào bảng, một lời cảnh báo về cấu trúc. Bốn nỗi đau không độc lập. Chuyển động nhanh sinh ra ảnh mờ; ảnh mờ làm mất bám; mất bám làm ta cần định vị lại và cần vòng đóng. Vì thế quy gán một con số APE cho đúng một nỗi đau là việc khó, và cách duy nhất làm được là quy gán theo từng khung. Hình 1.5 vẽ bản đồ ba tầng: nỗi đau, cơ chế hỏng, nhóm phương pháp.



Hình 1.5. Từ nỗi đau sang cơ chế hỏng sang nhóm phương pháp. Cột giữa là cột phải đo trước: nó nói *cái gì* hỏng, và chỉ khi biết điều đó thì cột phải mới có nghĩa. Hai mũi tên đứt bên trái nhắc rằng bốn nỗi đau nối nhau, nên một cải thiện ở nỗi đau đầu có thể làm ba nỗi đau còn lại nhẹ đi mà không cần đụng tới chúng.

A. Chuyển động nhanh

Cơ chế hỏng ở đây không phải “ảnh xấu”. Nó là một chuỗi rất cụ thể. Bộ bám chiều các điểm bản đồ vào khung mới bằng một tiên đoán pose, rồi tìm mô tả khớp trong một cửa sổ quanh vị trí chiều. Khi camera xoay nhanh, tiên đoán lệch nhiều hơn bán kính cửa sổ. Số tương ứng tụt, RANSAC còn ít điểm, và một loạt quyết định phía sau đổi theo.

Điều đó cho ra hai hướng chữa hoàn toàn khác nhau, và chúng nằm ở hai trụ khác nhau. Hướng thứ nhất là làm cho việc ghép cặp không cần tiên đoán tốt: ghép cặp học được kiểu SuperGlue hoặc LightGlue dùng ngữ cảnh toàn ảnh, và dòng quang dày kiểu RAFT [17] không cần cửa sổ nào cả. Hướng thứ hai là làm cho tiên đoán tốt hơn, bằng chính IMU mà bạn đã có.

Nhận định thẳng thắn của chương này: hướng thứ hai nên thử trước, vì nó rẻ hơn nhiều bậc. IMU 200 Hz cho một ước lượng xoay rất tốt trên khoảng thời gian 50 mili-giây. Nếu tiên đoán của bạn còn dùng mô hình vận tốc không đổi ở những đoạn xoay nhanh, thì bạn đang bỏ không một cảm biến đã có sẵn. Chỉ khi hướng đó đã cạn mới đáng trả giá độ trễ cho một bộ ghép cặp học được.

B. Ảnh mờ do chuyển động

Ảnh mờ tấn công đúng giả định nền của ORB: rằng một góc là một góc. Khi ảnh nhoè theo một hướng, đáp ứng góc sụp xuống, và cái còn lại giống một cạnh hơn một góc. Tệ hơn, mô tả đổi theo *hướng* nhoè, nên hai ảnh mờ theo hai hướng khác nhau của cùng một cảnh cho hai mô tả khác nhau.

Có bốn nhóm chữa, và chúng khác nhau cả về trụ lẫn về giá. Đặc trưng học được bền hơn với mờ ở mức nào đó, và đó là mảng của chương F/28. Khử mờ trước khi trích đặc trưng là một tầng cộng thêm, tốn đúng ngân sách mà bạn đang thiếu. Hiệu chỉnh chùm tia có ý thức về mờ mô hình hoá chính quá trình phơi sáng, đắt nhưng đúng về mặt vật lý. Camera sự kiện thì bỏ hẳn khái niệm phơi sáng. Ba nhóm sau nằm ở chương F/34.

Điều kiện đáng thử ở đây nghiêm ngặt hơn ta tưởng. Trước tiên phải đo: trên chuỗi của bạn, bao nhiêu phần trăm khung thật sự mờ tới mức làm hỏng việc, và bao nhiêu phần APE quy được về những khung đó. Nếu chỉ một phần rất nhỏ số khung bị mờ nặng và hệ thống vượt qua chúng nhờ IMU, thì cả bốn nhóm trên đều đang chữa một vết thương không chảy máu.

C. Đóng vòng — ví dụ mẫu của cổng 1

Đóng vòng là nỗi đau minh hoạ rõ nhất vì sao khung bốn cổng lại cần thiết. Nó có bốn tầng nối tiếp, và một vòng chỉ đóng khi qua được cả bốn:

1. **Lấy ứng viên** — tìm các keyframe cũ có thể là cùng một chỗ. Cổ điển dùng túi từ trên mô tả nhị phân.
2. **Ghép đặc trưng** — tìm tương ứng giữa khung hiện tại và ứng viên.
3. **Kiểm chứng hình học** — ước lượng Sim3 bằng RANSAC và đòi đủ số inlier.
4. **Tối ưu** — hiệu chỉnh đồ thị pose rồi hiệu chỉnh chùm tia toàn cục.

Bây giờ là chỗ mấu chốt. Phần lớn công trình học sâu về đóng vòng cải thiện tầng 1: NetVLAD [18] và các hậu duệ của nó, hoặc một pipeline phân cấp kiểu HF-Net [19]. Nếu tầng chết trên hệ của bạn không phải tầng 1, thì mọi cải thiện đó là công cốc, và bạn sẽ thấy đúng hiện tượng ở câu hỏi mở đầu: recall tăng mạnh, số vòng đóng không đổi.

I MẠCH 2 — chẩn đoán đóng vòng trước khi chọn phương pháp

- | | |
|---------------------|--|
| 1 Vấn đề | Trên chuỗi dài, vòng gần như không đóng. Triệu chứng dễ thấy nhất là quỹ đạo trôi mà không bao giờ được kéo về. |
| Giải pháp | Đếm số ứng viên sống sót sau <i>từng</i> tầng trong bốn tầng, trên vài chục lần đi qua cùng một chỗ. |
| Giả định | Bốn tầng nối tiếp và độc lập đủ để quy trách nhiệm cho một tầng. |
| Giá phải trả | một buổi thêm bộ đếm và một buổi chạy lại. |
| 2 Vấn đề | Bộ đếm đặt sai chỗ thì số ra vô nghĩa. Nếu bộ đếm nằm trước câu lệnh điều kiện, nó đếm cả những ứng viên chưa từng được xét. |
| Giải pháp | Đặt bộ đếm ngay sau mỗi điều kiện, và ghi cả lý do bị loại. |
| Hệ quả | Đây không phải chi tiết vụn vặt. Một lần đo nội bộ trên chính nhánh này bạn đầu quy sai thủ phạm chỉ vì bộ đếm nằm trước điều kiện, và kết luận đảo ngược sau khi sửa. |
| 3 Vấn đề | Giả sử phép đếm đúng cho thấy phần lớn ứng viên chết ở tầng 3, tức Sim3 RANSAC không hội tụ. |

Giải pháp	Khi đó tầng đáng đầu tư là tầng 2, vì RANSAC chết do quá ít tương ứng đúng. Ghép cặp học được ở chương F/28 chứa đúng chỗ đó.
Vấn đề mới	Nói ngưỡng inlier để RANSAC dễ hội tụ hơn là cá mồi lớn nhất và cũng là sai lầm đắt nhất, vì nó mua thêm vòng đóng bằng cách mua thêm vòng <i>sai</i> .
4 Vấn đề	Một vòng sai đắt hơn một vòng bị bỏ lỡ rất nhiều. Bỏ lỡ chỉ để lại độ trôi; đóng sai kéo lệch cả một đoạn bản đồ và gần như không cứu được.
Giải pháp	Chọn điểm vận hành lệch hẳn về phía precision, và giữ tầng kiểm chứng hình học kể cả khi tầng lấy ứng viên đã học được.
Giả định	Chi phí của hai loại lỗi bất đối xứng, và bạn ước lượng được mức bất đối xứng đó.

CẠM BẦY

Cạm bẫy hay gặp nhất khi đọc mẩu này là tin vào một metric của tầng 1 để quyết định cho cả bốn tầng. Recall@1 cao nghĩa là “ứng viên đúng có mặt trong danh sách”. Nó không nói gì về việc ứng viên ấy có đủ tương ứng để Sim3 hội tụ hay không. Hai đại lượng đó chỉ trùng nhau khi tầng 2 và tầng 3 dư sức, mà đó chính là điều bạn đang nghi ngờ. Cách phân biệt rõ nhất: cấp thẳng ứng viên đúng từ chân trị vị trí, rồi đếm xem bao nhiêu ứng viên hoàn hảo đó vẫn không đóng được vòng. Nếu con số đó lớn, tầng 1 không phải thủ phạm.

D. Trọng số cho IMU

Nỗi đau thứ tư ít được nói tới nhất, và theo đánh giá của chương này, nó là chỗ có tỉ số lợi ích trên rủi ro tốt nhất. Lý do rất cấu trúc: mọi thứ ở đây diễn ra *bên trong* bộ ước lượng cổ điển, nên không có khối nào bị xóa đi.

Cơ chế hỏng như sau. Trong đồ thị nhân tử, cạnh IMU mang một ma trận thông tin suy ra từ mật độ phổ nhiễu của con quay và gia tốc kế, lan truyền qua bước tiền tích phân [8]. Các con số đó đến từ bảng thông số cảm biến hoặc từ một lần hiệu chỉnh tĩnh. Chúng không mô tả nhiễu thật lúc thiết bị đang rung, đang nóng, hoặc đang chịu gia tốc lớn. Đặt trọng số quá cao thì bộ tối ưu bám theo IMU và bỏ qua ảnh; quá thấp thì IMU thành vô dụng và bạn mất tỉ lệ tuyệt đối cùng khả năng vượt qua vài khung mù.

Nhóm phương pháp hứa chữa gồm: học mô hình chuyển động quán tính trực tiếp từ chuỗi IMU, khử nhiễu con quay bằng mạng, và học hiệp phương sai theo từng phép đo thay vì dùng hằng số. TLIO [20] là mốc dễ đọc nhất của nhóm đầu. Tất cả nằm ở chương F/34.

Điều kiện đáng thử: bạn phải chỉ ra được rằng trọng số hiện tại *đang* sai. Phép thử rõ nhất là một oracle trên chính siêu tham số đó. Quét hệ số nhân của ma trận thông tin IMU trên một lưới thô, lấy kết quả tốt nhất cho từng chuỗi, và so với giá trị mặc định. Khoảng cách đó là trần đóng góp của mọi phương pháp học trọng số. Nếu nó nhỏ, cả nhóm bị loại ở cổng 1 và bạn tiết kiệm được nhiều tháng.

Bảng 1.3. Bản đồ bốn nỗi đau. Cột “điều kiện để đáng thử” là cột quyết định: nó là phép đo bạn phải chạy trước, và trong phần lớn trường hợp nó sẽ tự loại bớt ba trong bốn dòng.

Nỗi đau	Cơ chế hỏng và cách đo	Nhóm hứa chữa	Chương	Điều kiện để đáng thử
Chuyển động nhanh	Tiên đoán chiều lệch quá bán kính cửa sổ tìm. Đo: vận tốc góc so với số tương ứng còn lại, theo từng khung	Ghép cặp học được, dòng quang đây; hoặc tiên đoán tốt hơn bằng IMU	28, 30, 34	Đã dùng hết thông tin xoay của IMU cho tiên đoán mà số tương ứng vẫn tụt ở đúng các đoạn xoay nhanh
Ảnh mờ	Đáp ứng góc sụp, mô tả đổi theo hướng nhỏ. Đo: chỉ số nhỏ theo khung, đổi chiều với khung bị mất bám	Đặc trưng bền với mờ; khử mờ; hiệu chỉnh chùm tia có ý thức về mờ; camera sự kiện	28, 34	Tỉ lệ khung mờ nặng đủ lớn, và quy được một phần APE về đúng các khung đó
Đóng vòng	Một trong bốn tầng chết. Đo: đếm ứng viên sống sót sau từng tầng, bộ đếm đặt sau điều kiện	Nhận dạng địa điểm học được (tầng 1); ghép cặp học được (tầng 2); kiểm chứng mạnh hơn (tầng 3)	28, 29	Phép đếm chỉ đúng tầng mà nhóm đó chữa. Cấp ứng viên từ chân trị để loại trừ tầng 1 trước
Trọng số IMU	Ma trận thông tin đến từ bảng thông số, không mô tả nhiều lúc rung và lúc nóng. Đo: quét hệ số nhân trên lưới thô	Học nhiều và bias IMU; hiệp phương sai theo từng phép đo; khử nhiễu con quay	34	Khoảng cách giữa hệ số tốt nhất theo chuỗi và giá trị mặc định lớn hơn hẳn sản nhiễu

E. Thứ tự ưu tiên đề xuất, và lập luận đằng sau nó

Nếu buộc phải xếp thứ tự khi chưa có bảng trần đóng góp của riêng bạn, chương này đề xuất thứ tự sau, kèm lý do. Đây là phán đoán chứ không phải kết quả đo, nên hãy coi nó là điểm khởi hành và thay bằng số của bạn ngay khi có.

1. **Trọng số IMU trước.** Nó nằm hoàn toàn bên trong bộ ước lượng cổ điển, nên không phải xoá khỏi nào. Phép thử oracle của nó rẻ nhất trong bốn nỗi đau: chỉ là quét một siêu tham số. Và nếu trần lớn, cách chữa đầu tiên thậm chí chưa cần mạng nào.
2. **Đóng vòng thứ hai.** Chi phí một vòng sai rất cao, nhưng phép chẩn đoán bốn tầng lại rẻ và cho kết luận sắc. Nhiều khả năng nó chỉ ra rằng tầng cần chữa không phải tầng mà phần lớn paper chữa, và riêng kết luận đó đã tiết kiệm hàng tháng.
3. **Chuyển động nhanh thứ ba.** Hướng rẻ là dùng tốt hơn IMU đã có, và hướng đó nên cạn trước khi trả giá độ trễ cho ghép cặp học được.
4. **Ảnh mờ cuối cùng.** Không phải vì nó không quan trọng, mà vì nó khó quy trách nhiệm nhất và cách chữa đắt nhất. Nó cũng chông lên chuyển động nhanh, nên một phần lợi ích của nó sẽ tự đến khi bạn xử lý mục thứ ba.

Thứ tự này có một điểm chung đáng chú ý: nó xếp theo *chi phí của phép chẩn đoán*, không xếp theo mức đau. Lý do là ở giai đoạn này bạn đang mua thông tin chứ chưa mua cải thiện, và thông tin rẻ nhất nên mua trước.

1.5 Ba mức can thiệp (Three Levels of Intervention)

Giả sử một ứng viên đã qua bốn cổng. Còn một quyết định nữa, và nó quyết định phần lớn chi phí thật: cắm sâu tới mức nào. Có ba mức, và chúng khác nhau không phải ở độ khó viết code mà ở *thứ bạn phải từ bỏ*.

A. Mức 1 – bổ trợ

Mạng chỉ cấp thêm một tín hiệu; đường cổ điển vẫn chạy và vẫn quyết định. Ba ví dụ điển hình: một mặt nạ vật thể động chỉ dùng để *loại* phép đo; một hiệp phương sai học được chỉ dùng để *cân lại* phép đo đã có; một tiên đoán học được chỉ dùng để *thu hẹp* cửa sổ tìm.

Tính chất định nghĩa của mức này là: nếu mạng chết hoặc trả rác, hệ thống suy biến về đúng baseline cổ điển. Điều đó cho bạn hai thứ vô giá. Thứ nhất là một công tắc: bật và tắt để so sánh, trên cùng một lần chạy nếu muốn. Thứ hai là một trần thiết hại: điều tệ nhất có thể xảy ra là mất một ít phép đo tốt.

B. Mức 2 – thay một khối

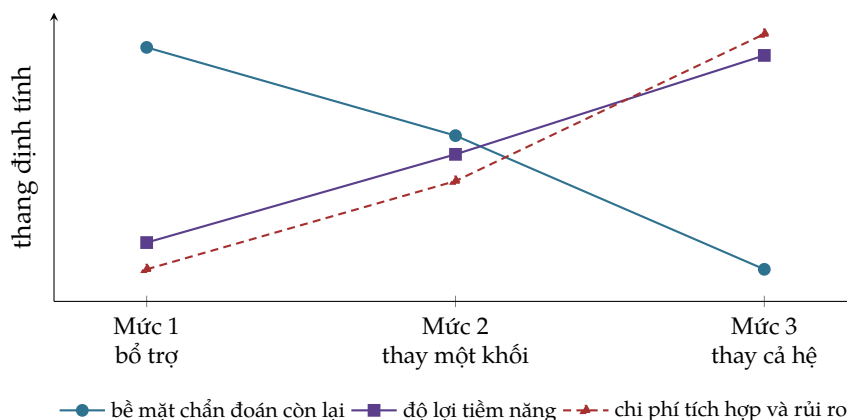
Mạng thay hẳn một khối cổ điển, nhưng giữ nguyên giao diện của khối đó. Trích đặc trưng học được thay ORB; nhận dạng địa điểm học được thay DBoW; độ sâu học được thay stereo. Backend không đổi một dòng.

Cái phải từ bỏ ở đây thường bị đánh giá thấp. Bạn không chỉ bỏ đoạn code cũ. Bạn bỏ luôn toàn bộ tinh chỉnh đã gắn với nó, bỏ tập các chế độ hỏng mà bạn đã thuộc lòng, và bỏ khả năng chạy song song hai đường để đối chứng lúc gỡ lỗi. Bề mặt chẩn đoán vẫn còn — vẫn có điểm, vẫn có tương ứng, vẫn có inlier — nhưng mỗi triệu chứng bây giờ có hai chỗ trú, và tách chúng ra cần một oracle chứ không cần một câu lệnh in.

C. Mức 3 – thay cả hệ

Mạng thay cả chuỗi ước lượng. Đây là trụ 3, và nó không phải một cuộc tích hợp mà là một cuộc chuyển hệ. Bạn được một hệ thống mới với các chế độ hỏng mới, và mất những thứ mà một hệ trưởng thành mang theo: quản lý bản đồ tăng dần, nhiều bản đồ, định vị lại, khả năng hỏi hiệp phương sai, và nhiều năm tinh chỉnh ngưỡng.

Nói thế không có nghĩa mức 3 luôn sai. Nó là lựa chọn đúng trong hai tình huống. Thứ nhất, khi bạn đang bắt đầu từ con số không trên một miền mà hệ đặc trưng thừa vốn đã không chạy nổi. Thứ hai, khi bạn cần bản đồ dày cho một mục đích khác và độ chính xác quỹ đạo chỉ là yêu cầu phụ.



Hình 1.6. Ba mức can thiệp trên một thang định tính, không phải thang đo. Điều đáng chú ý không phải ba đường mà là *khoảng cách* giữa chúng: độ lợi tăng gần tuyến tính, còn chi phí và rủi ro tăng nhanh hơn, trong khi bề mặt chẩn đoán — số đại lượng trung gian bạn còn kiểm tra được bằng một kỳ vọng vật lý — sụt gần như đứng ở mức 3.

D. Quy tắc chọn mức, và vì sao nó là quy tắc một chiều

Quy tắc rất gọn: **không lên mức tiếp theo cho tới khi đã đo được rằng mức hiện tại bị chặn dưới mức bạn cần**. Nó nghe hiển nhiên nhưng gần như không ai làm, vì mức cao hơn luôn hấp dẫn hơn về mặt trí tuệ.

Có một lý do sâu hơn để đi từ dưới lên: **tính đảo ngược**. Mức 1 gỡ ra bằng một cờ cầu hình. Mức 2 gỡ ra trong vài ngày nếu bạn còn giữ đường cũ. Mức 3 thì không gỡ ra được, vì sau sáu tháng thì mọi thứ quanh nó đã được xây theo nó. Chi phí thật của mức 3 không phải chi phí tích hợp mà là chi phí của một quyết định không thu hồi được.

Bảng 1.4. Ba mức can thiệp. Cột “cách quay lại” là cột hay bị bỏ qua nhất khi lên kế hoạch, và nó là cột định giá rủi ro thật của quyết định.

Mức	Cơ chế	Phải bỏ đi cái gì	Cách quay lại	Hỏng ra sao
1. Bổ trợ	Mạng cấp thêm tín hiệu; đường cổ điển vẫn quyết định	Không bỏ gì; chỉ thêm một tầng lọc hoặc một trọng số	Một cờ cầu hình	Mất một ít phép đo tốt; hệ suy biến về baseline
2. Thay một khối	Mạng thay khối cổ điển, giữ nguyên giao diện với backend	Đoạn code cũ, phần tinh chỉnh gắn với nó, và tập chế độ hỏng đã quen	Vài ngày, nếu còn giữ đường cũ chạy song song	Triệu chứng có hai chỗ trú; cần oracle để tách; hỏng im lặng nếu mạng không biết nói “không biết”
3. Thay cả hệ	Mạng thay cả chuỗi ước lượng, không còn bước hình học tường minh	Quản lý bản đồ, nhiều bản đồ, định vị lại, hiệp phương sai, nhiều năm tinh chỉnh	Gần như không; sau vài tháng mọi thứ quanh nó đã xây theo nó	Không còn đại lượng trung gian để hỏi; sửa lỗi nghĩa là huấn luyện lại

CẦN NHỚ

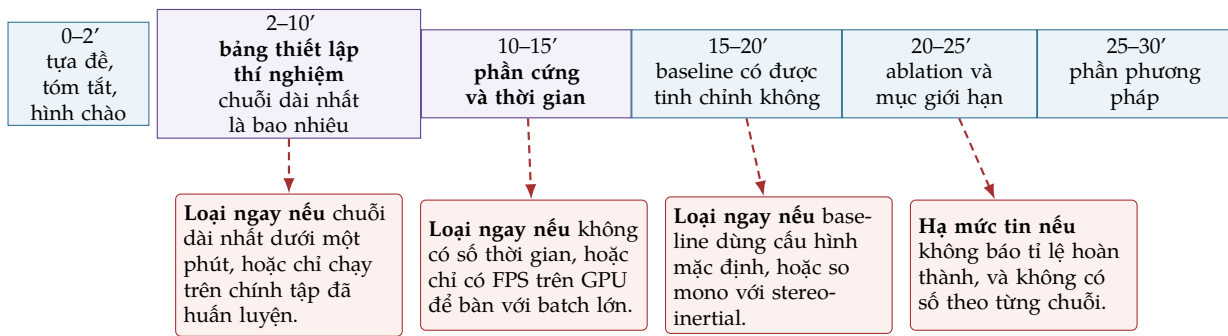
Hai câu quyết định phần lớn các lựa chọn trong Phần F. Thứ nhất: đo trần đóng góp của một khối *trước*, rồi mới đi tìm phương pháp thay khối đó — nếu một khối hoàn hảo cũng chỉ cải thiện được vài phần trăm thì cả nhánh nghiên cứu đó đã bị chặn trên. Thứ hai: trong một vòng phản hồi, một phép đo sai mà im lặng đắt hơn nhiều một phép đo thiếu mà lên tiếng, nên hãy ưu tiên các can thiệp chỉ *trừ đi* phép đo hoặc chỉ *cân lại* chúng. [T1]

1.6 Đọc một paper SLAM học sâu trong ba mươi phút (Reading a Deep SLAM Paper in 30 Minutes)

Bốn công và ba mức là khung để quyết định. Mục này là kỹ năng thao tác hằng ngày: mở một paper, và trong ba mươi phút biết nên bỏ hay nên đọc kỹ. Chương E/23-doc-paper đã dạy đọc paper nói chung theo lối ba lượt [21]. Ở đây thứ tự bị đảo lại có chủ ý, vì với mảng này thông tin quyết định nằm ở phần thí nghiệm chứ không ở phần phương pháp.

A. Thứ tự đọc, và vì sao nó ngược

Lý do đảo thứ tự rất đơn giản. Phần phương pháp cho bạn biết tác giả *định* làm gì. Phần thí nghiệm cho bạn biết điều đó đã được kiểm chứng tới đâu, và với mảng SLAM học sâu thì khoảng cách giữa hai thứ ấy thường rất lớn. Một phương pháp thanh lịch chỉ được thử trên chuỗi trong nhà dài một phút không nói gì về hệ của bạn.



Hình 1.7. Thứ tự đọc ba mươi phút, đảo ngược so với thứ tự trình bày của paper. Hai ô tím là hai ô đắt nhất về mặt thông tin và rẻ nhất về mặt thời gian, nên chúng đứng trước. Bốn ô đỏ là các cổng loại: chạm vào một trong ba ô đầu thì dừng đọc, khỏi mở phần phương pháp.

B. Sáu chặng, và câu hỏi của từng chặng

- **0–2 phút, định vị.** Phương pháp này ở trụ nào trong năm trụ, và nó đòi mức can thiệp nào? Chỉ hai câu đó thôi đã đủ để biết bạn phải bỏ cái gì nếu nhận nó.
- **2–10 phút, bảng thiết lập thí nghiệm.** Tập nào, chuỗi nào, và quan trọng nhất: *chuỗi dài nhất họ chạy dài bao nhiêu*. Rất nhiều kết quả ấn tượng của mảng bản đồ neural đến từ các cảnh tổng hợp trong nhà, mỗi cảnh cỡ hai nghìn khung. Một chuỗi EuRoC ngoài đời dài vài phút, và một lần chạy sản phẩm dài hàng chục phút. Cũng ở chặng này: dữ liệu huấn luyện của họ có chứa tập đánh giá của bạn không.
- **10–15 phút, phần cứng và thời gian.** GPU nào, độ trễ hay thông lượng, batch bao nhiêu, và phép đo có tính cả tiền xử lý lẫn chuyển dữ liệu không. Nếu không có mục nào nói về thời gian, hãy coi như phương pháp chưa được thiết kế cho thời gian thực.
- **15–20 phút, baseline.** Họ so với ORB-SLAM3 phiên bản nào, cấu hình nào, có bật đóng vòng không, đơn mắt hay stereo-inertial. So một hệ stereo-inertial học được với một baseline đơn mắt là phép so sánh phổ biến và vô nghĩa.
- **20–25 phút, ablation và mục giới hạn.** Mục giới hạn viết thật là dấu hiệu chất lượng mạnh nhất mà bạn có. Nếu paper có một đoạn nói thẳng phương pháp gây ở đâu, hãy tăng mức tin cậy cho toàn bộ phần còn lại.
- **25–30 phút, phương pháp.** Và chỉ đọc với đúng một câu hỏi trong đầu: *ý tưởng thiết kế nào còn giá trị kể cả khi tôi không dùng hệ này*. Đó thường là thứ duy nhất bạn mang về được.

C. Ba dấu hiệu bên ngoài bài báo

Ba việc kiểm tra sau tốn dưới năm phút và thường quyết định nhiều hơn cả bài báo. Thứ nhất, có mã nguồn chạy được không, và lần cập nhật gần nhất là bao giờ. Thứ hai, mục theo dõi lỗi của kho mã: nếu nhiều người báo cùng một vấn đề tái lập mà không có trả lời, đó là dữ liệu. Thứ ba, có ai độc lập với nhóm tác giả chạy lại được chưa. Trong mảng SLAM, khoảng cách giữa “chạy được trên máy tác giả” và “chạy được trên máy bạn” đặc biệt rộng, vì phụ thuộc vào phiên bản thư viện, dữ liệu và hiệu chỉnh cảm biến.

D. Ba câu hỏi bạn phải tự trả lời sau ba mươi phút

Kết thúc một buổi đọc bằng ba câu, viết ra giấy, không giữ trong đầu. Một: phương pháp này chạm nổi đau nào trong bốn nổi đau, và nó chữa *tầng* nào của nổi đau đó. Hai: nếu nhận nó, tôi phải xoá cái gì khỏi hệ hiện tại. Ba: bằng chứng cụ thể nào sẽ làm tôi đổi kết luận, và bao giờ tôi nên xem lại. Câu thứ ba biến một kết luận “chưa đáng” thành một kết luận có hạn dùng, thay vì thành một định kiến.

E. Bốn câu hỏi mang sang bảy chương sau

Cả Phần F được viết theo đúng bốn câu hỏi dưới đây, và bạn nên đọc mọi phương pháp bằng chúng. Chúng là bản rút gọn của cả chương này thành một khuôn dùng lại được.

1. **Nó giải quyết việc gì?** Phát biểu bằng một thất bại cụ thể của hệ cổ điển, có chuỗi và có tần suất. “Học đặc trưng tốt hơn” không phải câu trả lời; “giữ được tương ứng ở các đoạn xoay nhanh, nơi hệ hiện tại tụt xuống dưới ngưỡng inlier” thì có.
2. **Cơ chế nào làm được điều đó?** Một đoạn, nói đúng thiết kế cốt lõi. Nếu bạn không tóm được cơ chế trong một đoạn thì hoặc paper viết chưa rõ, hoặc bạn chưa hiểu, và cả hai trường hợp đều nên dừng lại.
3. **Có đáng làm không?** Đây là bốn cổng: chi phí độ trễ trên Orin, lượng dữ liệu cần, cách nó hỏng khi ra ngoài phân phối, và — câu hay bị quên nhất — *cái gì trong hệ cổ điển phải bị bỏ đi để nhận nó*.
4. **Nếu đi sâu thì học được gì từ thiết kế của nó?** Bài học còn giá trị kể cả khi bạn không dùng phương pháp. Đây thường là thứ duy nhất bạn mang về được, và cũng là lý do đọc tiếp bảy chương sau dù phần lớn phán quyết là “chưa đáng”.

Câu thứ ba là câu mà chương này phục vụ. Câu thứ tư là câu giữ cho việc đọc không biến thành một chuỗi từ chối, và nó là lý do bảy chương sau vẫn đi vào cơ chế chứ không dừng ở kết luận.

1.7 Giới hạn và trường hợp hỏng

Khung trong chương này có giới hạn riêng, và một chương bộ lọc mà không tự soi thì chính nó thành cái bẫy.

Nó lệch về phía từ chối. Bốn cổng tối ưu cho việc không phí công, chứ không tối ưu cho việc không bỏ lỡ. Một khung loại hết mọi thứ cũng là một khung hỏng, chỉ là hỏng theo cách khó thấy. Thuốc giải là giữ một danh sách theo dõi ngắn: mỗi ứng viên bị loại kèm đúng một dòng ghi bằng chứng nào sẽ lật kết luận, rồi chạy lại khung mỗi năm một lần.

Trần đóng góp giả định các khối tách rời được. Thay một khối bằng chân trị làm đổi phân phối đầu vào của khối đứng ngay sau. Trong một hệ có vòng phản hồi giữa bản đồ và quyết định keyframe, “đóng góp của khối X” không còn là một đại lượng xác định. Đây đúng là giới hạn đã nêu ở chương E/24, và ở SLAM nó nặng hơn vì vòng phản hồi chặt hơn.

Phép tính ngân sách giả định pipeline nối tiếp và bị chặn bởi độ trễ. Nếu CPU đang là nút thắt còn GPU rảnh, một mạng có thể gần như miễn phí về thời gian tường. Ngược lại, nếu bạn đã đẩy trích đặc trưng lên GPU thì phép cộng đơn giản ở hình 1.3 sẽ cho kết luận sai theo hướng lạc quan.

Bốn nỗi đau không độc lập. Quy một con số APE về đúng một nỗi đau đòi hỏi quy gán theo từng khung, mà việc đó chưa có phương pháp gọn. Trong thực tế bạn sẽ phải sống với những kết luận kiểu “phần lớn sai số tập trung ở một đoạn vài chục giây, nơi có cả xoay nhanh lẫn ảnh mờ”, và chấp nhận rằng tách hai nguyên nhân đó cần một thí nghiệm riêng.

Mọi phán quyết trong Phần F đều có hạn dùng. Chúng chốt tại tháng 9/2026 và nói về *trạng thái bằng chứng*, không nói về giới hạn của cái khả dĩ. Mô hình nền hình học đang chuyển động nhanh, và một kết luận “chưa đáng” của hôm nay hoàn toàn có thể sai trong mười hai tháng.

Khung này được hiệu chỉnh cho người đã có hệ thống. Nếu bạn đang bắt đầu từ đầu, trên một cảm biến hoặc một miền mà hệ đặc trưng thừa vốn không chạy nổi, thì thứ tự ưu tiên đảo lại: một hệ đầu-cuối có thể là con đường nhanh nhất tới thứ chạy được, và tính chẩn đoán được là món xa xỉ bạn mua sau.

1.8 Thực hành

Repo và tài nguyên. Dựng lại nền so sánh trước khi thử bất cứ thứ gì: [UZ-SLAMLab/ORB_SLAM3](#) làm baseline, [MichaelGrupp/evo](#) cho APE và RPE, và một harness chạy lập có ghi seed. Để thử nhanh phía học được mà không phải viết lại pipeline: [cvv/Hierarchical-Localization](#) cho tuyến đặc trưng và nhận dạng địa điểm, [cvv/glue-factory](#) cho ghép cặp, [kornia](#) cho các phép hình học khả vi. Phía triển khai: TensorRT và các công cụ đo trên Jetson. Bản đồ hướng của mảng này thì xem kho [changh95/visual-slam-roadmap](#). Danh sách tên mô hình dẫn đầu chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; hạ tầng như COLMAP, evo, TensorRT thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Với hệ hiện tại của bạn, oracle nào nên chạy *đầu tiên*, và nó chặn trên cái gì? Nếu recall@1 của tầng lấy ứng viên tăng mạnh mà số vòng đóng không đổi, ba cách giải thích là gì và phép thử nào tách được chúng? Một mạng báo một tốc độ khung hình rất cao trên GPU để bàn thì con số đó nói được gì — và không nói được gì — về Orin ở batch bằng 1, và bạn sẽ đo phần nó không nói bằng cách nào? Một can thiệp mức 1 và một can thiệp mức 2 cùng hứa cải thiện như nhau: chọn cái nào và vì sao?

Mini project. Hai khối dự án ngay dưới đây là phần thực hành chính của chương; chạy chúng theo thứ tự, vì khối thứ hai dùng lại số đo của khối thứ nhất.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Trụ 1 — frontend nhận thức	Mạng sinh ra phép đo: điểm, mô tả, tương ứng, ứng viên đóng vòng. Giao diện với backend không đổi, nên đây là trụ rủi ro thấp nhất và chín nhất.
••• Trụ 2 — backend tối ưu	Mạng không đổi phép đo mà đổi <i>trọng số</i> của nó: hiệp phương sai học được, mô hình nhiễu IMU, hoặc bộ tối ưu khả vi. Nằm hoàn toàn bên trong bộ ước lượng cổ điển.
••• Trụ 3 — hệ đầu-cuối	Ảnh vào, pose ra, không còn bước hình học tường minh. Được tính bền, mất toàn bộ đại lượng trung gian để chẩn đoán.
•• Trụ 4 — hiểu cảnh	Mạng quyết định phép đo nào được phép vào, thường bằng cách loại điểm trên vật thể động. Nó chỉ trừ đi, nên mạng chết thì hệ về đúng hành vi cổ điển.
•• Trụ 5 — mô hình nền và bản đồ neural	Hai thứ khác nhau bị gộp tên: tiên nghiệm học sẵn cho độ sâu và đặc trưng, và việc đổi bản đồ thành trường liên tục hay tập splat. Cái đầu nhắm độ bền phép đo, cái sau nhắm chất lượng ảnh dựng lại.
••• Trần đóng góp	Mức cải thiện tối đa của một khối, đo bằng cách thay khối đó bằng chân trị rồi chạy cả hệ. Nếu trần là c thì mọi phương pháp cho khối đó đều bị chặn trên bởi đúng c , bất kể bảng của họ đẹp tới đâu — và c phải do bạn đo, không do ai báo.
••• Sàn nhiễu	Độ lệch chuẩn giữa các lần chạy cùng một cấu hình. Nó là đơn vị đo của mọi so sánh; cải thiện nhỏ hơn nó thì không xác nhận được, nên ứng viên đó trượt cổng 2.
••• Tỷ lệ hoàn thành chuỗi	Phần trăm khung hệ thống còn bám được. Phải báo cùng APE, vì một hệ mất bám giữa chừng chỉ được chấm trên phần dễ và vì thế có APE đẹp giả.
•• Batch bằng 1	Chế độ chạy bắt buộc của SLAM thời gian thực: mỗi lần đúng một khung. GPU chỉ đạt hiệu suất cao ở batch lớn, nên FPS trong paper thường lệch rất xa thực tế của bạn, xa tới mức không dùng để kết luận được.
••• Hông im lặng	Khối vẫn trả về đầu ra trông bình thường, kèm điểm tin cậy bình thường, trong khi đầu ra đó sai. Nguy hiểm vì SLAM là vòng phản hồi: phép đo sai ở lại trong bản đồ.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Bề mặt chẩn đoán	Số đại lượng trung gian mà bạn còn kiểm tra được bằng một kỳ vọng vật lý: số inlier, sai số chiều ngược, hiệp phương sai. Nó sụt gần như dựng đứng khi lên mức can thiệp 3.
●●● Bốn tầng của đóng vòng	Lấy ứng viên, ghép đặc trưng, kiểm chứng hình học bằng Sim3 RANSAC, rồi tối ưu. Một vòng chỉ đóng khi qua đủ bốn, nên trước khi chọn phương pháp phải biết tầng nào đang chết.
●● Ba mức can thiệp	Bổ trợ (mạng chỉ cấp thêm tín hiệu), thay một khối (giữ nguyên giao diện), thay cả hệ. Chúng khác nhau chủ yếu ở thứ phải từ bỏ và ở khả năng quay lại.
●●● Vì sao đọc bảng thí nghiệm trước phần phương pháp?	Vì phần phương pháp nói tác giả <i>định</i> làm gì, còn bảng thí nghiệm nói điều đó đã được kiểm chứng tới đâu. Với mảng này khoảng cách giữa hai thứ rất lớn, nên đọc ngược tiết kiệm phần lớn thời gian.
●●● Vì sao không suy ngân sách Orin từ FPS trong paper?	Vì FPS là số thông lượng, đo ở batch lớn với dữ liệu sẵn trên GPU. SLAM chạy batch bằng 1, trả thêm chi phí chuyển dữ liệu, chia bộ nhớ với bản đồ, và chậm dần khi máy nóng.
●●● Vì sao vòng đóng sai đắt hơn vòng bỏ lỡ?	Bỏ lỡ chỉ để lại độ trôi, còn sửa được sau. Đóng sai kéo lệch cả một đoạn bản đồ và gần như không cứu được, nên điểm vận hành phải lệch hẳn về phía precision.
●● Vì sao đi từ mức 1 lên, không nhảy thẳng mức 3?	Vì tính đảo ngược. Mức 1 gỡ bằng một cờ, mức 2 gỡ trong vài ngày, mức 3 thì sau vài tháng mọi thứ quanh nó đã xây theo nó. Chi phí thật của mức 3 là chi phí của một quyết định không thu hồi được.
●●● Recall@1 tăng mạnh mà số vòng đóng không đổi — nguyên nhân?	Tầng chết không phải tầng lấy ứng viên. Thường là Sim3 RANSAC không hội tụ vì quá ít tương ứng đúng. Phân biệt: cấp ứng viên đúng từ chân trị vị trí rồi đếm xem bao nhiêu ứng viên hoàn hảo vẫn không đóng được vòng.
●●● APE đẹp bất thường trên chuỗi khó — nghĩa là gì?	Nghi ngay mắt bắm giữa chừng: hệ chỉ được chằm trên đoạn dễ. Phân biệt bằng tỉ lệ hoàn thành chuỗi; nếu nó dưới 100% thì con số APE kia không so sánh được với ai.
●● Mạng löt cồng 3 lúc thử nhưng bỏ khung sau mười phút — nguyên nhân?	Nhiệt và chế độ điện. Chu kỳ trôi khi máy nóng, nên số đo trong ba mươi giây đầu không phải số đo lúc vận hành. Phân biệt: đo lại phân vị 95 sau mười phút chạy liên tục ở đúng chế độ điện.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
hloc [19]	Trụ 1 ở mức 2: thay cả tầng lấy ứng viên lẫn tầng ghép cặp bằng khối học được, giữ nguyên phần kiểm chứng hình học	Bảng chứng rằng mức 2 đã chín cho định vị ngoại tuyến; chưa phải cho vòng bám thời gian thực
DynaSLAM [10] và các nhánh cùng ý tưởng	Trụ 4 ở mức 1: mặt nạ ngữ nghĩa chỉ <i>loại</i> phép đo, không thêm gì vào bộ tối ưu	Ví dụ sạch nhất của can thiệp đảo ngược được bằng một cờ
Kimera [11]	Trụ 4: tầng ngữ nghĩa dựng <i>cạnh</i> bộ ước lượng hình học chứ không thay nó	Kiến trúc đáng học kể cả khi không dùng phần ngữ nghĩa
DROID-SLAM [9]	Trụ 3 ở mức 3: xoá chuỗi đặc trưng thừa, giữ lại một lõi tối ưu khả vi	Mốc tham chiếu của mức 3; ngân sách GPU vượt xa Orin theo cấu hình gốc
MASt3R-SLAM [13]	Trụ 5: mô hình nền hình học làm nguồn tương ứng và độ sâu cho một vòng SLAM	Rất mới; ở đây chỉ nhắc ý tưởng, không dẫn con số
TLIO [20]	Trụ 2: học mô hình quán tính và độ bất định của nó thay vì lấy từ bảng thông số cảm biến	Nhóm gần nhất với nỗi đau trọng số IMU

MINI PROJECT SLAM · Bảng trần đóng góp bốn khối — đo trước khi đọc thêm paper nào · 2 ngày

Mục tiêu	biến câu hỏi “nên đầu tư vào đâu” thành một bảng bốn dòng có số, để ba tháng tới đi vào đúng một chỗ.
Dữ liệu	ba chuỗi EuRoC với độ khó khác nhau, trong đó phải có một chuỗi bạn biết là hay hỏng; cấu hình mặc định của chính nhánh bạn.
Các bước	trước hết đo sàn nhiều: chạy cùng cấu hình mười lần mỗi chuỗi, ghi APE cùng tỉ lệ hoàn thành, lấy độ lệch chuẩn. Rồi chạy bốn biến thể oracle. Một: dùng pose chân trị chiều ngược để loại mọi tương ứng sai, đo trần của ghép cặp. Hai: cấp ứng viên đóng vòng từ khoảng cách vị trí chân trị, đo trần của tầng lấy ứng viên. Ba: quét hệ số nhân của ma trận thông tin IMU trên lưới thô rồi lấy giá trị tốt nhất mỗi chuỗi, đo trần của việc học trọng số. Bốn: bỏ mọi khung có chỉ số nhỏ vượt ngưỡng, đo phần APE quy được về ảnh mờ.
Xong khi	bạn có một bảng bốn dòng, mỗi dòng ghi mức cải thiện trần kèm sai số, và bạn khoanh được đúng những dòng có trần lớn hơn hai lần sàn nhiều. Kiểm tra chéo: tổng bốn trần phải lớn hơn khoảng cách giữa hệ của bạn và một hệ hoàn hảo; nếu nhỏ hơn thì có một nguồn sai số thứ năm bạn chưa gọi tên.
Nối với	E/24 cho công thức cỡ mẫu và cho khái niệm oracle; bảng này là đầu vào bắt buộc của khối dự án tiếp theo và của cả bảy chương sau.

MINI PROJECT SLAM · Ngân sách Orin, đo trước khi viết dòng code tích hợp nào · 1 ngày

Mục tiêu	biến cổng 3 từ một phỏng đoán thành một con số, và biết trước một ứng viên có lọt hay không.
Dữ liệu	chính Jetson Orin của bạn, ở đúng chế độ điện dùng khi vận hành; một chuỗi EuRoC chạy được ít nhất mười phút liên tục.
Các bước	lập hồ sơ thời gian luồng bám theo từng khối: trích đặc trưng, ghép cặp, RANSAC, tối ưu pose, tiền tích phân IMU. Ghi cả trung vị lẫn phân vị 95, vì phân vị mới là thứ làm bỏ khung. Tính phần ngân sách còn trống so với trần 50 mili-giây tại 20 Hz. Rồi lấy một mạng ứng viên, chuyển sang TensorRT ở độ chính xác nửa, và đo độ trễ ở batch bằng 1 trên đúng máy đó. Đo hai lần: lúc máy nguội và sau mười phút chạy liên tục.
Xong khi	bạn phát biểu được một câu có ba con số: “luồng bám đang tiêu X mili-giây ở phân vị 95, còn trống Y , mạng tốn Z ở batch 1 sau khi máy nóng — do đó GO hoặc NO-GO”. Kiểm tra chéo: chênh lệch giữa số lúc nguội và lúc nóng phải khác không; nếu bằng nhau thì bạn chưa đo đúng chế độ điện.
Nối với	D/21-inference-va-trien-khai cho phần chuyển đổi và lượng tử hoá; B/11-gioi-han-tinh-toan cho lý do vì sao batch bằng 1 lại xa peak tới vậy; chương F/34 cho phần đo lại cho trung thực trên phần cứng nhúng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Bạn đã cải thiện tầng không hỏng. Vòng đóng có bốn tầng nối tiếp, và nhận dạng địa điểm chỉ chạm tầng đầu. Lẽ ra phải đếm số ứng viên sống sót sau *từng* tầng trước, với bộ đếm đặt sau câu lệnh điều kiện chứ không phải trước.
- A2. Ba câu: chuỗi dài nhất họ chạy dài bao nhiêu, đo trên phần cứng nào ở batch bao nhiêu, và baseline có được tinh chỉnh không. Câu thứ nhất loại được nhiều paper nhất, vì nó trả lời được trong ba phút bằng cách nhìn đúng một bảng.
- A3. Vì SLAM là vòng phản hồi còn phân lớp ảnh thì không. Một dự đoán sai của bộ phân lớp làm sai đúng một mẫu. Một phép đo sai trong SLAM đi vào bản đồ, rồi bản đồ quyết định phép đo tiếp theo; với một vòng đóng sai thì cả đoạn quỹ đạo lệch vĩnh viễn.
- A4. Vì đó là số thông lượng ở batch lớn, còn bạn chạy batch bằng 1 và bị chặn bởi độ trễ. Thêm vào đó: chi phí chuyển dữ liệu, GPU có thể đã bận với phần trích đặc trưng, bộ nhớ dùng chung với bản đồ, và chu kỳ trôi khi máy nóng. Cách duy nhất là đo trên chính Orin của bạn.

- A5. Bảng trần đóng góp. Thay lần lượt từng khối bằng chân trị rồi đo cả hệ; dòng nào có trần lớn hơn hai lần sàn nhiễu thì đáng, còn lại thì không. Bảng đó chạy trong hai ngày và nó quyết định ba tháng.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn có một bếp ăn chạy tốt nhiều năm. Bạn biết từng vị trí, biết ai chậm ở đâu, biết khi nào món ra trễ thì phải nhìn chỗ nào. Một người bán hàng mang tới cái máy thái rau rất nhanh, đẹp hơn tay người, và có clip chứng minh.

Trước khi mua, bạn hỏi bốn câu. Một: món ra trễ có thật sự vì khâu thái rau không, hay vì cái bếp lò? Cách biết duy nhất là bấm giờ từng khâu trong một tuần. Hai: nếu mua, tôi nhận ra cái thiện bằng gì, khi mà mỗi tối bếp lại đông khác nhau? Ba: cái máy có vừa mặt bàn và vừa cầu dao không, lúc mọi thứ khác cũng đang bật? Bốn: khi gặp một loại rau nó chưa từng thấy, nó dừng lại hay vẫn đẩy ra một mớ trông giống rau đã thái?

Câu thứ tư là câu đắt nhất. Một người thái rau sẽ dừng và hỏi. Cái máy thì không biết dừng, và thứ nó đẩy ra sẽ đi thẳng vào nồi. Trong một cái bếp mà món này nấu tiếp từ món kia, một sai lầm im lặng lan ra khắp bữa ăn.

Rồi còn chuyện lắp sâu tới đâu. Đặt cái máy cạnh bàn cũ thì hôm nào hỏng bạn rút phích. Bỏ hẳn bàn cũ thì phải chờ mua bàn mới. Đập cả bếp đi xây lại quanh cái máy thì không quay về được nữa. Cái giá không nằm ở tiền mua máy. Cái giá là mỗi cái máy lắp thêm là một chỗ bạn không còn nhìn vào bên trong được, khi món ra sai mà không hiểu vì sao.

Chỗ không rút gọn được: chỗ bốn nỗi đau chồng lên nhau. Nói “chứa ảnh mờ thì sai số giảm” nghe gọn nhưng sai, vì đoạn ảnh mờ cũng là đoạn xoay nhanh, và xoay nhanh mới là thứ làm mất bám. Chia phần trách nhiệm giữa hai nguyên nhân cùng xảy ra một lúc thì hiện chưa có cách làm gọn, và mọi cách nói đơn giản về nó đều đang giấu bước đó đi.

Một câu: Trước khi hỏi mạng nào tốt hơn, hãy đo xem khối nào trong hệ của bạn đang thật sự làm hỏng việc, và bạn còn bao nhiêu mili-giây để trả cho nó.

1.9 Kết nối

B/03-giai-phau-he-thong là tiền đề trực tiếp: năm trụ chỉ là câu hỏi “độ khó đã bị đẩy đi đâu” đặt cho một hệ SLAM, và ranh giới giữa các khối chính là chỗ một mạng chen vào được. E/24-thi-nghiem-va-ablation cấp hai công cụ mà cổng 1 và cổng 2 dùng lại nguyên vẹn, là oracle experiment và sàn nhiễu. B/07-chia-du-lieu-va-danh-gia là nơi đọc ý nghĩa của các con số, đặc biệt phần precision và recall cho tầng lấy ứng viên đóng vòng. B/08-dich-chuyen-phan-phoi là mặt lý thuyết của cổng 4: nó giải thích vì sao độ chính xác tụt khi ra ngoài phân phối, còn chương này quan tâm chuyện khác — vì sao bạn không biết là nó đã tụt. B/11-gioi-han-tinh-toan và D/21-inference-va-trien-khai là hai chương phải mở khi làm cổng 3.

Về phía trước, bảy chương còn lại của Phần F đều dùng khung này. F/28 và F/30 là trụ 1. F/29 là nỗi đau đóng vòng, và nó bắt đầu đúng bằng phép đếm bốn tầng của mạch 2. F/31 là trụ 2 và trụ 3. F/32 là trụ 5, đọc với cảnh giác rằng chất lượng ảnh dựng lại không phải độ chính xác quỹ đạo. F/33 là trụ 4. F/34 gom hai nỗi đau còn lại, trọng số IMU và ảnh mờ, cộng phần đưa lên phần cứng. Cuối cùng, E/26-lo-trinh-hoc cho biết nên đọc bảy chương đó theo thứ tự nào tùy mục tiêu của bạn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao trần đóng góp phải đo trước khi đọc paper, chứ không phải sau khi đã chọn được phương pháp ứng viên?

2. Một can thiệp mức 1 và một can thiệp mức 2 hứa cùng mức cải thiện: bạn chọn cái nào, và lập luận của bạn dựa vào đại lượng nào chứ không dựa vào độ chính xác?
3. Điều gì làm một phép đo sai mà im lặng đắt hơn hẳn trong SLAM so với trong một bài toán phân lớp ảnh, và cơ chế nào của hệ thống gây ra sự khác biệt đó?
4. Bốn tầng của đóng vòng có thể chết ở tầng nào, và với mỗi tầng thì nhóm phương pháp nào chữa đúng chỗ — vì sao chọn nhầm tầng lại cho ra hiện tượng “metric tăng mà kết quả không đổi”?
5. Phép tính ngân sách ở cổng 3 hỏng trong trường hợp nào, và sai lệch của nó đi theo hướng lạc quan hay bi quan?
6. Nếu bốn nỗi đau không độc lập thì việc quy một phần APE về đúng một nỗi đau còn có nghĩa không, và bạn sẽ phát biểu kết luận theo dạng nào để vẫn dùng được?
7. Khung bốn cổng lệch về phía từ chối: cơ chế nào trong quy trình làm việc của bạn ngăn nó biến thành một định kiến lâu dài?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.